Лекція №12 (2 години). Робота з DOM та Event Loop.

План лекції

1. Що таке DOM (Document Object Model).
2. Пошук та вибір елементів на сторінці (Query Selectors).
3. Маніпуляції з DOM: зміна тексту, HTML та атрибутів.
4. Створення, додавання та видалення елементів.
5. Обробка подій (Event Listeners) та Об'єкт події (Event Object).
6. Спливання подій (Event Bubbling) та Делегування.
7. Однопотоковість JS та концепція Event Loop (Цикл подій).

Перелік умовних скорочень

- **DOM** (Document Object Model) — об'єктна модель документа.
- **API** (Application Programming Interface) — інтерфейс прикладного програмування.
- **JS** — JavaScript.
- **HTML** — HyperText Markup Language.

Вступ

До цього моменту ми розглядали JavaScript як ізольовану мову програмування: вивчали змінні, функції та масиви. Проте в браузері головне завдання JS — це взаємодія з веб-сторінкою. Нам потрібно мати можливість прочитати те, що користувач ввів у форму, змінити колір кнопки при кліку, показати вспливаюче вікно або плавно додати новий товар у кошик.

Для цього браузер надає JavaScript'у спеціальний інструмент — **DOM (Document Object Model)**. Це своєрідний "міст" між HTML-кодом та JS. Окрім того, користувачі взаємодіють зі сторінкою непередбачувано: клікають мишкою, скролять, натискають клавіші. Щоб обробляти ці дії послідовно і не "підвісити" браузер, JavaScript використовує унікальний механізм керування чергою завдань, який називається **Event Loop**.

Мета цієї лекції — навчитися "оживляти" HTML-документ за допомогою JS, реагувати на дії користувача та розібратися, як браузер виконує код під капотом.

1. Що таке DOM?

Браузер отримує від сервера звичайний текстовий HTML-файл. Перш ніж намалювати сторінку, він читає цей текст і будує в оперативній пам'яті його внутрішнє представлення у вигляді **дерева**. Це дерево і називається DOM.

У цьому дереві кожен HTML-тег стає **об'єктом** (який називають вузлом — Node). Наприклад, тег `<body>` стає об'єктом з властивостями, `<p>` всередині нього стає дочірнім об'єктом і так далі.

Головним "входом" у це дерево є глобальний об'єкт **`document`**. Майже всі маніпуляції зі сторінкою починаються зі звернення до об'єкта `document`.

2. Пошук елементів на сторінці

Щоб змінити якийсь елемент, спочатку його треба знайти у DOM-дереві і зберегти у змінну. У сучасному JS для цього використовують два основних методи, які працюють точно так само, як і CSS-селектори.

2.1. `document.querySelector(selector)`

Шукає і повертає **перший** знайдений елемент, який відповідає CSS-селектору. Якщо нічого не знайдено — повертає `null`.

```
// Знайти елемент за класом
const btn = document.querySelector(".submit-btn");

// Знайти елемент за ID (хоча для ID є спеціальний метод getElementById)
const header = document.querySelector("#main-header");

// Складний CSS-селектор
const firstListItem = document.querySelector(".menu > li:first-child");
```

2.2. `document.querySelectorAll(selector)`

Шукає і повертає **всі** елементи, що відповідають селектору. Зверніть увагу: він повертає структуру `NodeList` (псевдомасив). Її не можна просто так змінити (наприклад `items.style.color = 'red'` — помилка). Потрібно перебирати її циклом `forEach`.

```
// Знайти всі параграфи в статті
const paragraphs = document.querySelectorAll("article p");
```

```
paragraphs.forEach((p) => {
  p.style.color = "gray"; // Змінюємо колір кожного знайденого параграфа
});
```

3. Маніпуляції з елементами

Після того, як ми знайшли елемент і поклали його в змінну, ми отримуємо доступ до його властивостей, які можна читати або перезаписувати.

Зміна тексту та HTML:

- `element.textContent` — читає або записує лише безпечний текст (без HTML-тегів).
- `element.innerHTML` — дозволяє вставити в елемент повноцінний HTML-код (наприклад `<br>` або `<b>`), який браузер відрендерить. *Небезпечно використовувати для вставки даних, отриманих від користувача (може призвести до XSS атак).*

```
const title = document.querySelector("h1");
title.textContent = "Новий заголовок!";
```

Зміна стилів: Будь-яка CSS властивість доступна через об'єкт `style`. Важливо: якщо в CSS властивість пишеться через дефіс (`background-color`), то в JS використовується camelCase (`backgroundColor`).

```
btn.style.backgroundColor = "red";
```

Керування класами (`classList`): Це найправильніший спосіб стилізації. Замість того, щоб міняти inline-стилі (`.style`), краще додавати або забирати CSS-класи, заздалегідь написані у файлі стилів.

- `element.classList.add('active')` — додає клас.
- `element.classList.remove('hidden')` — видаляє клас.
- `element.classList.toggle('dark-mode')` — перемикач: додає клас, якщо його немає, і видаляє, якщо є.

4. Створення, додавання та видалення елементів

DOM дозволяє динамічно створювати нові HTML-теги, яких не було в початковому документі.

```
// 1. Створюємо новий елемент в ПАМ'ЯТІ (він ще не на сторінці)
const newDiv = document.createElement("div");
newDiv.textContent = "Я щойно народився!";
newDiv.classList.add("alert");

// 2. Знаходимо "батька" на сторінці
const container = document.querySelector(".container");

// 3. Вставляємо новий елемент ('дітям' в кінець)
container.appendChild(newDiv);
// Альтернатива: container.prepend(newDiv) - вставить на початок
```

Видалення:

```
const oldAd = document.querySelector(".advertisement");
oldAd.remove(); // Елемент назавжди зникає зі сторінки
```

5. Обробка подій (Events)

Сайт має реагувати на дії користувача: кліки, рух мишки, введення тексту, відправку форми. Це називається **подіями (Events)**. Для того, щоб "слухати" події, до DOM-елемента потрібно "причепити ухо" — Event Listener.

Синтаксис: `element.addEventListener(ім'я_події, функція_обробник)`

```
const myBtn = document.querySelector("#clickMe");

// Коли станеться подія 'click', JS викличе передану стрілочну функцію
myBtn.addEventListener("click", () => {
  alert("Кнопку натиснуто!");
});
```

Популярні події: `click`, `mouseenter`, `mouseleave`, `keydown`, `submit` (для форм), `input` (для полів вводу).

Об'єкт події (Event Object): Щоразу, коли стається подія, браузер автоматично створює об'єкт з усіма деталями цієї події і передає його першим аргументом у нашу функцію (зазвичай його називають `e` або `event`).

```
document.addEventListener("keydown", (event) => {
  console.log(`Ви натиснули клавішу: ${event.key}`);
  // event.key містить літеру (наприклад "Enter" або "A")
});
```

6. Спливання подій (Bubbling) та Делегування

Уявіть, що у вас є кнопка всередині `<div>`, який лежить у `<body>`. Якщо ви клікаєте на кнопку, подія `click` насправді відбувається не лише на кнопці. Подібно до бульбашки повітря під водою, подія кліку утворюється на найменшому елементі (кнопці) і починає "спливати" вгору до предків: на `<div>`, потім на `<body>`, аж до `document`.

Це називається **Спливання (Event Bubbling)**.

Завдяки спливанню ми можемо застосувати потужний патерн — **Делегування подій**. Уявіть, що у вас є список з 1000 товарів (тегів `<li>`), і кожен треба обробляти по кліку. Вішати 1000 Event Listeners на кожен `<li>` — це катастрофа для пам'яті комп'ютера. Натомість, ми вішаємо ОДИН обробник на їхнього спільного батька (наприклад, `<ul>`), і коли користувач клікає на будь-який `<li>`, подія спливає до `<ul>`, де ми її і перехоплюємо!

```
const list = document.querySelector("ul.products");

list.addEventListener("click", (event) => {
  // event.target — це той найменший елемент, на який ДІЙСНО клікнули
  if (event.target.tagName === "LI") {
    event.target.style.backgroundColor = "yellow";
  }
});
```

7. Event Loop (Цикл подій)

Це найулюбленіше питання на співбесідах рівня Middle.

JavaScript — однопотоковий (Single-threaded). Це означає, що він має лише одного "робітника", який може виконувати лише **одне завдання одночасно**. Але що робити, якщо нам потрібно зробити паузу (наприклад, таймер `setTimeout` на 5 секунд), або завантажити величезну фотографію з сервера? Якби робітник просто чекав 5 секунд, то вся сторінка зависла б (кнопки перестали б натискатися, скрол би зламався).

Для вирішення цієї проблеми браузер надає допоміжну екосистему — Web API та **Event Loop**.

Як це працює крок за кроком:

1. **Call Stack (Стек викликів):** Це місце, де робітник (JS) виконує код. Код виконується рядок за рядком. Якщо він бачить звичайну функцію (наприклад `console.log`), він виконує її негайно.
2. **Web API:** Якщо код зустрічає асинхронне завдання (наприклад, таймер `setTimeout` або запит до бази даних), він НЕ чекає! Він аутсорсить (віддає) це завдання самому браузеру (у Web API) і каже браузеру: "Коли закінчиш рахувати 5 секунд, поклади цю функцію у чергу".
3. **Task Queue (Черга завдань):** Місце збору функцій, які "чекають", поки їх виконають.
4. **Event Loop (Вічний цикл):** Це спеціальний механізм (сторож), який постійно виконує одну перевірку: *«Чи повністю порожній зараз Call Stack?»*
   - Якщо в стеку ще виконується якийсь код — Event Loop просто чекає.
   - Як тільки Call Stack стає АБСОЛЮТНО пустим, Event Loop бере перше завдання з Черги і відправляє його в Стек на виконання.

```
console.log("1. Я виконуюсь першим");

setTimeout(() => {
  console.log("2. Я виконаюсь третім! Хоча таймер 0 секунд");
}, 0);

console.log("3. Я виконуюсь другим");
```

Чому `setTimeout` виконався останнім, хоча час очікування 0 секунд? Тому що `setTimeout` — це асинхронне Web API. Його callback-функція була відкинута в Web API, одразу ж перемістилася в Чергу Завдань (Task Queue). Але **Event Loop не має права пустити функцію з черги в Стек, доки Стек не спорожніє**. Тому спочатку виведуться 1 і 3 (синхронний код у стеку), Стек звільниться, і лише тоді Event Loop закине функцію з таймера (2) на виконання.

> *Наступна лекція пролле світло на інший вид черги — Мікротаски (Microtasks), куди потрапляють Проміси (Promises). Мікротаски завжди мають вищий пріоритет над звичайними Макротасками (наприклад `setTimeout`).*

Висновки

1. DOM — це дерево об'єктів у пам'яті браузера, створене на основі HTML.
2. Сучасний спосіб пошуку елементів у DOM реалізується через `querySelector` (повертає один елемент) та `querySelectorAll` (повертає колекцію).

3. Змінювати зовнішній вигляд динамічних компонентів найкраще за допомогою маніпуляцій з класами через `classList` (а не напряму через стилі `style`).
4. Метод `addEventListener` дозволяє підписатися на події. Завжди використовуйте його замість застарілих атрибутів `onclick` у HTML.
5. Спливання подій (Bubbling) лежить в основі оптимізаційного патерну "Делегування подій", коли замість 1000 обробників на дітях ставиться один обробник на їхньому батьку.
6. Event Loop — це архітектурний механізм браузера, який дозволяє однопотоковому JavaScript виконувати асинхронні задачі, не "підвішуючи" сторінку.
7. Асинхронний код (навіть з таймером 0) ніколи не долучиться до виконання, поки синхронний код (основний потік програми) повністю не завершить свою роботу.

Джерела

1. MDN Web Docs: Introduction to the DOM
2. Javascript.info: Документ (DOM)
3. Javascript.info: Введення в події
4. Philip Roberts: What the heck is the event loop anyway? (JSConf EU) — легендарне відео про Event Loop.

Запитання для самоперевірки

1. Що таке DOM-дерево і чим воно принципово відрізняється від статичного HTML-файлу?
2. Який метод потрібно використати, якщо ви хочете знайти всі кнопки з класом `.btn` на сторінці? Який тип даних він поверне?
3. В чому небезпека використання властивості `innerHTML` для вставки даних, які ввів користувач? Яка властивість безпечніша?
4. Чому для зміни зовнішнього вигляду елемента за допомогою JS рекомендується використовувати методи конструкції `classList`, а не пряму зміну властивості `style`?
5. Напишіть код, який створить новий тег `<li>` з текстом "Яблуко" і додасть його в кінець списку `<ul>` з класом `.fruits`.
6. Опишіть своїми словами, що таке "спливання подій" (Bubbling). Чому об'єкт події (`event`) такий важливий у патерні "Делегування"?
7. Уявіть, що JS виконує складний математичний цикл `for` на мільярд ітерацій (синхронний код). Що станеться зі сторінкою? Чому?
8. Що таке Call Stack (Стек викликів) та Task Queue (Черга завдань), і яку роль між ними відіграє Event Loop?